MACHINE LEARNING PROJECT

Customer Churn Prediction

Axmedov Yusufjon 202438404



Libraries to Import

Core data, visualization, preprocessing, model, metric, and persistence dependencies used in the notebook.

PYTHON

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pickle

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import classification_report
```

Load & Inspect the Dataset

The checks establish row count, schema preview, and one unique customer identifier per record.

PYTHON

```
df = pd.read_csv('dqlab_telco_final.csv')

df.shape
df.head()
df.customerID.nunique()
```

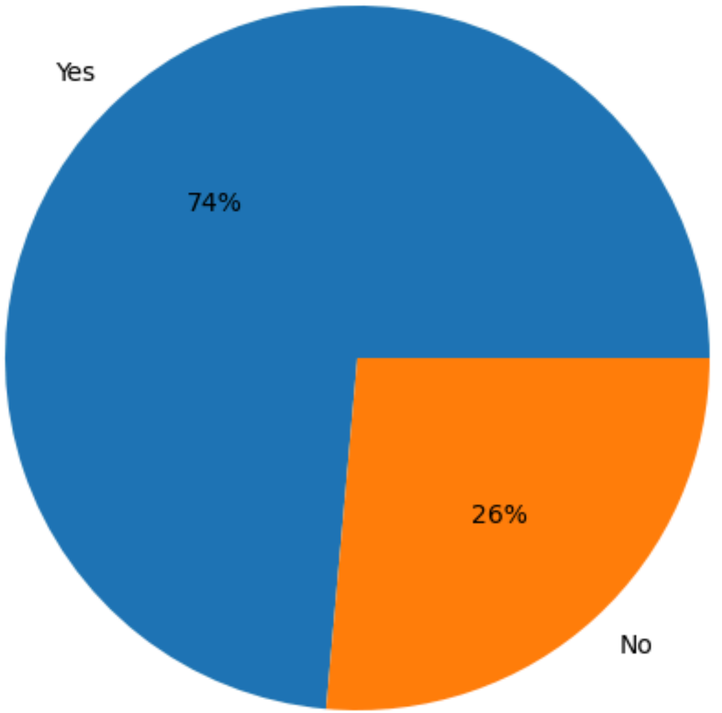
verified output → shape: (6950, 13) | unique customerID: 6950

Exploratory Data Analysis

The notebook uses EDA to inspect class distribution and the proportion of customers who churn.

PYTHON

```
fig = plt.figure(figsize=(10, 5))
ax = fig.add_subplot(121)
df['Churn'].value_counts().plot(kind='pie',
   autopct='%1.1f%%', ax=ax)
ax.set_title('Percentage of Churn Customer')
ax.set_ylabel('')
plt.show()
```



Remove Unnecessary Columns

customerID and UpdatedAt identify or timestamp records; the notebook removes both before modeling.

PYTHON

```
cleaned_df = df.drop(['customerID', 'UpdatedAt'], axis=1)
cleaned_df.head()
```

Encode & Prepare Features

The LabelEncoder loop converts object columns to numeric values. The verified describe() output summarizes 6,950 rows across 11 prepared columns.

PYTHON

```
le = LabelEncoder()
for column in cleaned_df.columns:
    if cleaned_df[column].dtype == 'object':
        cleaned_df[column] = le.fit_transform(cleaned_df[column])

cleaned_df.describe()
```

Train/Test Split

Predictors and target are separated, then split into training and testing subsets.

PYTHON

```
X = cleaned_df.drop('Churn', axis=1)
y = cleaned_df['Churn']

x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

x_train.shape
y_train.shape
y_train.value_counts(normalize=True)
```

verified output → x_train (4865, 10) | y_train (4865,) | churn: 0.734841 / 0.265159

Model 1 — Logistic Regression

The notebook fits a baseline classifier and evaluates its predictions on the training data.

PYTHON

```
log_model = LogisticRegression()
log_model.fit(x_train, y_train)

y_train_pred = log_model.predict(x_train)
print(classification_report(y_train, y_train_pred))
```

Model 2 & 3 — Ensemble Methods

The notebook compares Gradient Boosting with Random Forest. Only the shown Gradient Boosting training and prediction path is reproduced here.

PYTHON

```
gb_model = GradientBoostingClassifier()
gb_model.fit(x_train, y_train)

y_train_pred = gb_model.predict(x_train)
y_test_pred = gb_model.predict(x_test)

print(classification_report(y_train, y_train_pred))
print(classification_report(y_test, y_test_pred))
```

Model Comparison & Final Model

Reported notebook result: Logistic Regression reaches 80% training accuracy and 79% testing accuracy, then is persisted as the selected model.

PYTHON

```
log_model = LogisticRegression()
log_model.fit(x_train, y_train)

pickle.dump(log_model, open('best_model_churn.pkl','wb'))
```

notebook-reported accuracy → training: 80% | testing: 79%

Conclusion & Source

The notebook demonstrates an end-to-end churn workflow: inspect, clean, encode, split, train, compare, and save. Its selected Logistic Regression model is reported at 80% training and 79% testing accuracy.

```
# Notebook caution
# Model experiments are needed before settling on a model.
```

SOURCE
Kaggle notebook by samran98
Customer Churn Prediction using Machine Learning